

CCSAR

Instructions: Language of the Computer Part II

Arquiteturas de Computadores para Sistemas Críticos
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Assembler Pseudo-instructions

- **Pseudo-instructions are shorthand or aliases for one or more actual instructions in the RISC-V ISA**
 - They **do not exist in the ISA** and are not directly executable by the CPU
 - Converted into one or more real RISC-V instructions by the Assembler
- **Simplify assembly programming**
 - Provide easier, more readable ways to perform common tasks
 - Abstract away details of the hardware implementation, allowing focus on logic rather than instruction encoding
 - Hiding details from the higher level is **another example of the great idea of abstraction**

Common Pseudo-Instructions

Pseudo-inst.	Meaning	Translation
<code>li rd, imm</code>	Load immediate value into <code>rd</code>	<code>addi (or)</code> <code>lui + addi</code>
<code>mv rd, rs</code>	Move value from <code>rs</code> to <code>rd</code>	<code>addi rd, rs, 0</code>
<code>nop</code>	No operation	<code>addi x0, x0, 0</code>
<code>la rd, label</code>	Load address of <code>label</code> into <code>rd</code>	<code>auipc rd, label[31:12]</code> <code>addi rd, rd, label[11:0]</code>
<code>not rd, rs</code>	Bitwise NOT of <code>rs</code> into <code>rd</code>	<code>xori rd, rs, -1</code>
<code>neg rd, rs</code>	Negate <code>rs</code> into <code>rd</code>	<code>sub rd, x0, rs</code>
<code>j label</code>	Unconditional jump to <code>label</code>	<code>jal x0, label</code>
<code>jal label</code>	Jump and link to <code>label</code>	<code>jal ra, label</code>
<code>ret</code>	Return from function	<code>jalr x0, 0(ra)</code>

Example: The `li` Pseudo-instruction

- The purpose of `li` is to provide a convenient way to load an immediate (constant) value into a register

```
li rd, imm
```

- `rd` is the destination register
 - `imm` is the constant value to load
- The translation of the `li` pseudo-instruction depends on the size of the immediate value

Example: The `li` Pseudo-instruction

- If the immediate **value fits in 12 bits** (from -2048 to 2047), `li` is translated into a single `addi` instruction
- Consider this example, with a positive immediate

```
li x5, 1024    # 0x400 in hexadecimal
```

- Translates to:

```
addi x5, x0, 1024
```

Example: The `li` Pseudo-instruction

- And now, with a negative immediate value

```
li x5, -500      # 0xFFFFFE0C in 32-bit, with sign extension
```

- Translates to:

```
addi x5, x0, -500
```

- This works because the `addi` instruction performs sign extension on the 12-bit immediate value
 - Ensuring that negative numbers are correctly handled

Example: The `li` Pseudo-instruction

- For immediate **values that do not fit** within a 12-bit signed range the assembler must use multiple instructions
 - The lower 12-bit immediate is sign-extended, so the upper 20 bits may need to be adjusted
- If the lower 12 bits are positive

```
li x5, 0x1F2345          # 0001 1111 0010 0011 0100 0101 (21 bits)
```

- Translates to:

```
lui x5, 0x1F2          # Load upper 20 bits (0x1F2 << 12) into x5
addi x5, x5, 0x345      # Add lower 12 bits (0x345) to x5
```

Example: The `li` Pseudo-instruction

- If the lower 12 bits are negative

```
li x5, 0x12345800      # 0001 0010 0011 0100 0101 1000 0000 0000
```

- Translates to:

- The upper field is incremented by 1 to compensate for sign extension of the lower 12-bit immediate

```
lui x5, 0x12346      # Adjust upper 20 bits (0x12345 + 1 = 0x12346)
addi x5, x5, 0x800    # 0x800 interpreted as signed value (-2048)
```

Takeaway

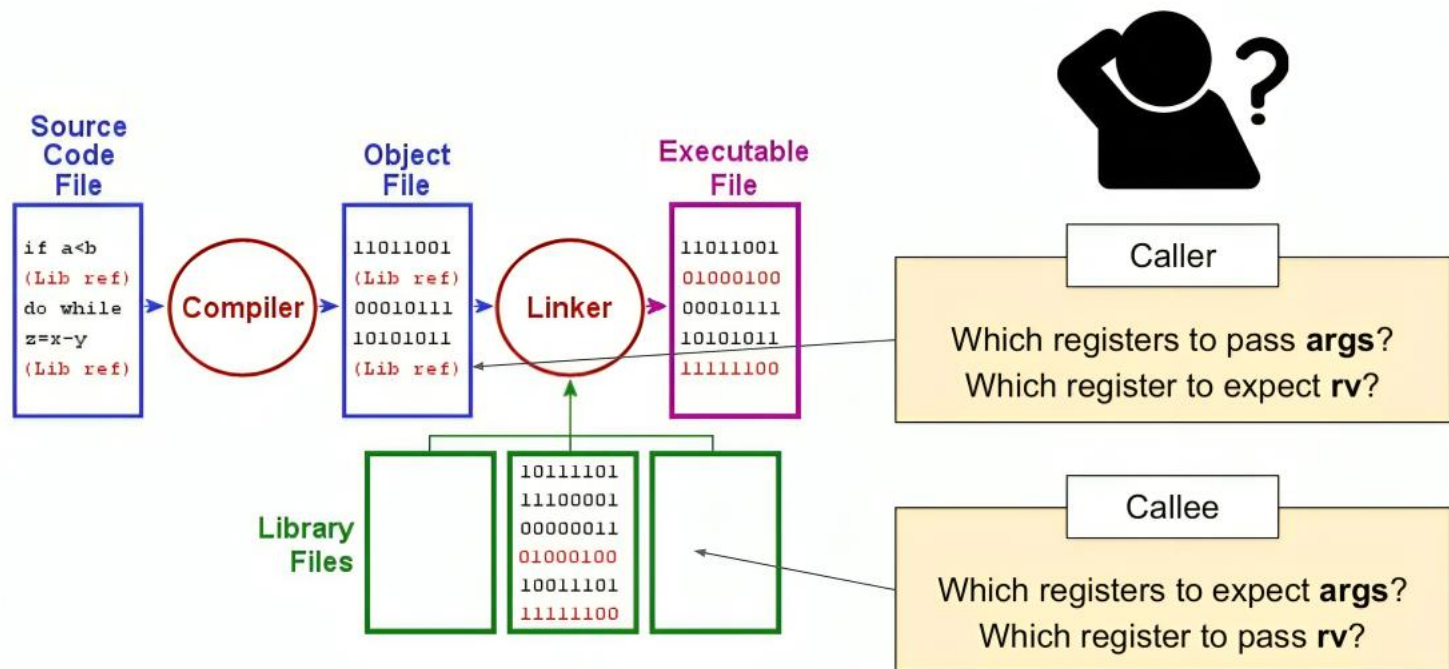
- Pseudo-instructions are not real instructions but are **convenient shortcuts**
- Are **translated** by the assembler **into one or more real instructions**
- Simplify programming by hiding low-level details
- Enhance readability and maintainability of assembly code

RISC-V Calling Conventions

- Machine instructions implement the mechanisms, but the choices are determined by designers
- One crucial aspect of RISC-V programming is the **calling convention**
 - Which governs how functions are called, how arguments are passed, and how values are returned
- In this lecture, we delve into the **calling conventions** defined by the **RISC-V ABI**
 - Assembly programming relies heavily on conventions
 - So, if you do not strictly follow conventions, you will not have working code (unless you write literally all the assembly you ever use)

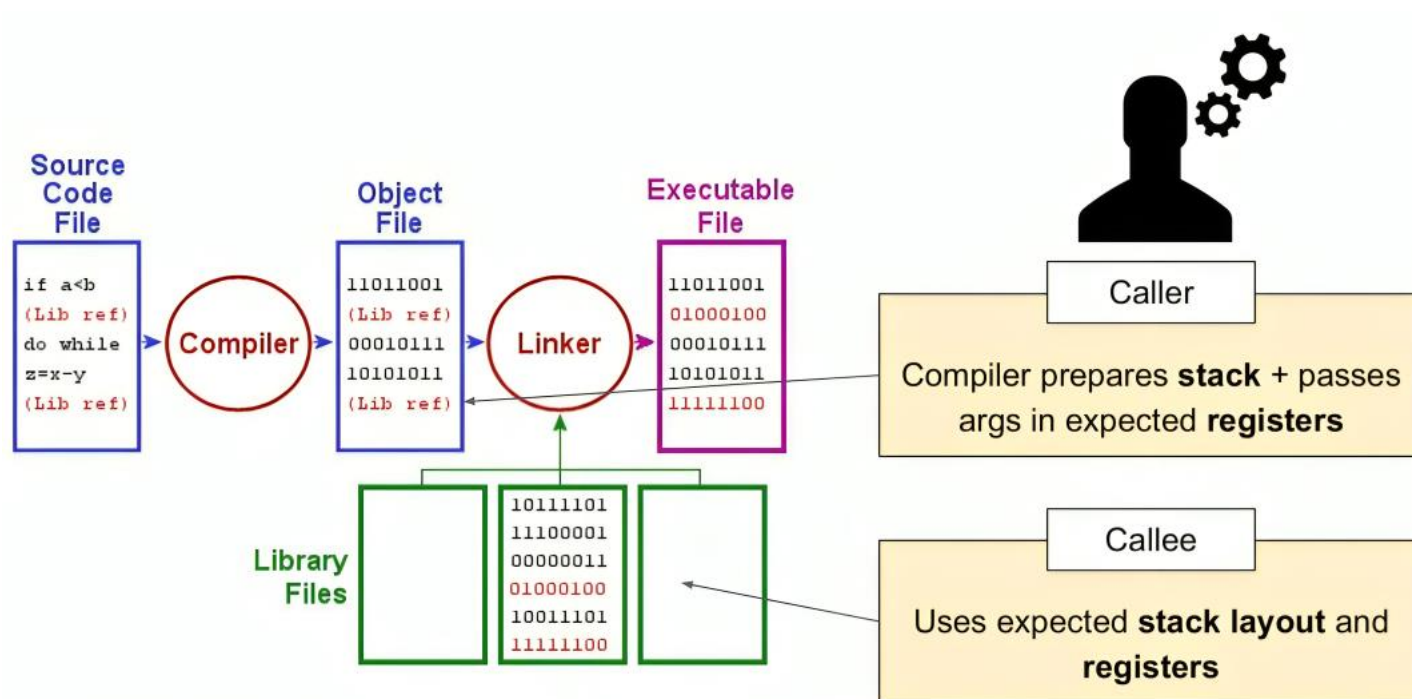
Without Conventions

- Each programmer makes their own choices
 - No common rules → no interoperability



With Conventions

- Everyone follows the same rules
 - Common rules → interoperable code



The RISC-V ABI

- The RISC-V Application Binary Interface **defines how software interacts at a binary level** on the RISC-V architecture
 - Crucial for the standardization and compatibility of software on the RISC-V architecture
- **Key components of the RISC-V ABI:**
 - Function calls and returns
 - Argument and return-value passing
 - Register usage
 - Stack layout and alignment
 - Data representation and alignment

Address Space

■ Heap

- Memory dynamically allocated during runtime

■ Stack

- Temporary data for handling procedure calls
- `%8` (frame pointer) and `%2` (stack pointer)

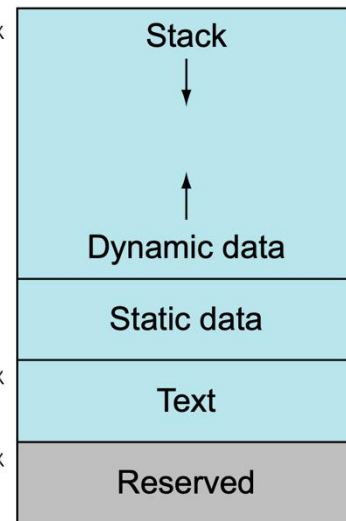
- In a typical process address space, the heap grows upward and the stack grows downward

SP → 0000 003f ffff fff0_{hex}

0000 0000 1000 0000_{hex}

PC → 0000 0000 0040 0000_{hex}

0



Address Space

■ Text

- Program code

■ Static data

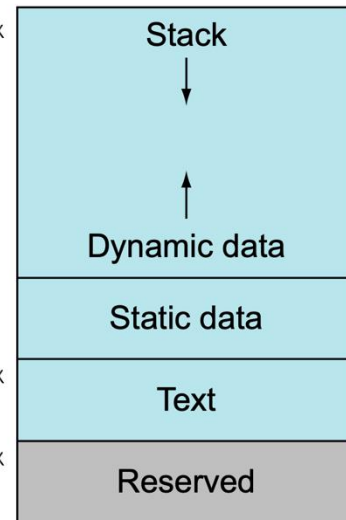
- Global variables
- Static variables in C, constant arrays and strings
- x3 (global pointer) initialized to address allowing $\pm$ offsets into this segment

SP → 0000 003f ffff fff0_{hex}

0000 0000 1000 0000_{hex}

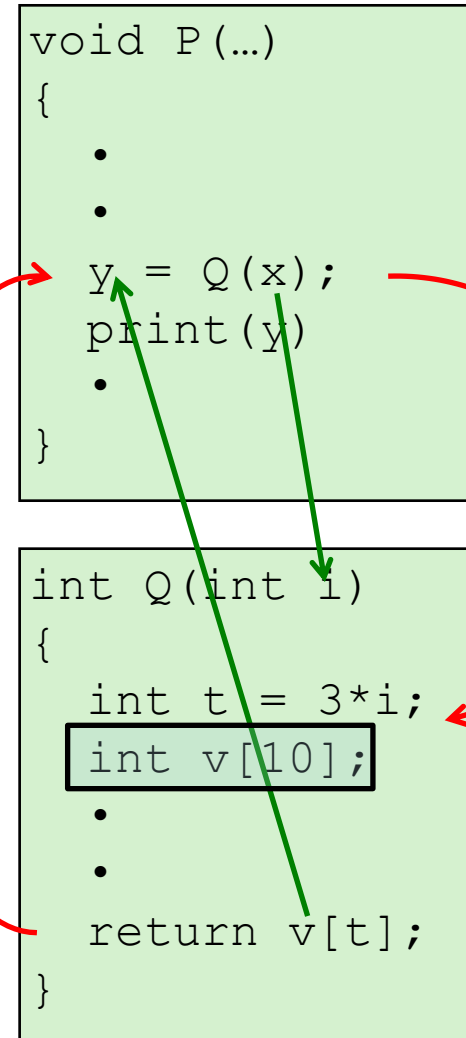
PC → 0000 0000 0040 0000_{hex}

0



Mechanisms in Procedures

- **Passing control**
 - To beginning of procedure code
 - Back to return point
- **Passing data**
 - Procedure arguments
 - Return value
- **Memory management**
 - Allocate during procedure execution
 - Deallocate upon return
- **These mechanisms are implemented with register and stack support**



Procedure Calling

- The execution of a procedure call happens when one procedure (**the caller**) invokes another procedure (**the callee**)
- In general, the execution of a procedure call follows six steps:
 1. Place parameters in registers (up to 8, additional via stack)
 2. Transfer control to procedure
 3. Acquire storage for procedure
 4. Perform procedure's operations
 5. Place result in register for caller
 6. Return to place of call

Procedure Calling

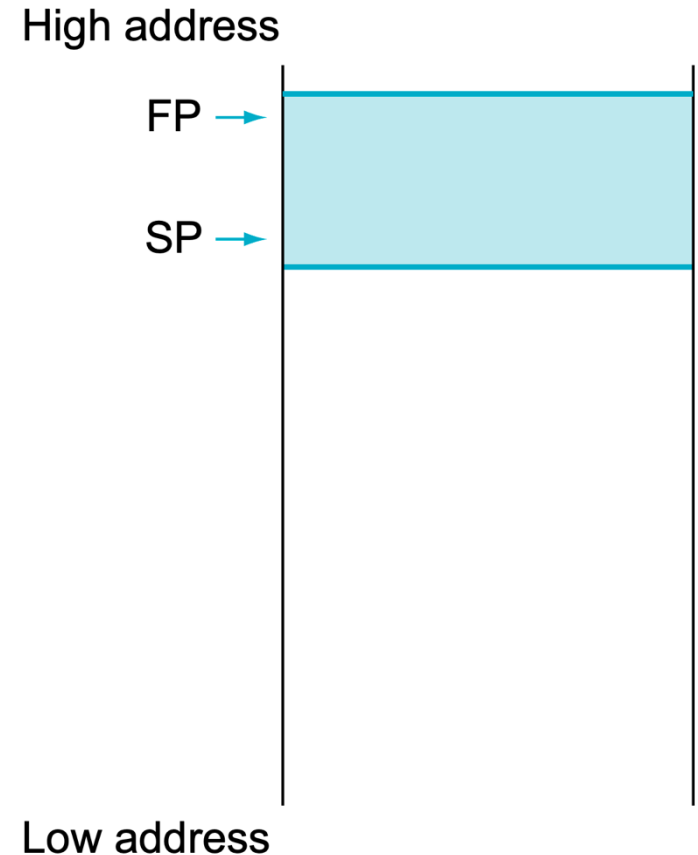
- **How can we ensure that a procedure call does not change data that is outside its scope?**
- **Programmers who write code in a high-level language never see the details of how one procedure calls another**
 - The compiler takes care of the low-level details
- **Programmers who write code in assembly must explicitly implement every procedure call and return**
 - The caller may have to save data before calling the callee
 - The callee may have to save data before running its operations

The Stack

- The bookkeeping associated with procedure calls is done in the **stack segment** around blocks of memory called **procedure frames**
- Register **x8/ƒp** (**frame pointer**) may be used as a frame pointer
 - Provides a stable reference within the current frame when a dedicated frame pointer is used
- Register **x2/sp** (**stack pointer**) points to the current top of the stack
 - As **sp** changes within a procedure, offsets to local data may change

The Stack

- The stack grows from **higher to lower** addresses
- **Increasing** the stack pointer **releases** stack space
- **Decreasing** the stack pointer **allocates** stack space



Memory Alignment Restrictions

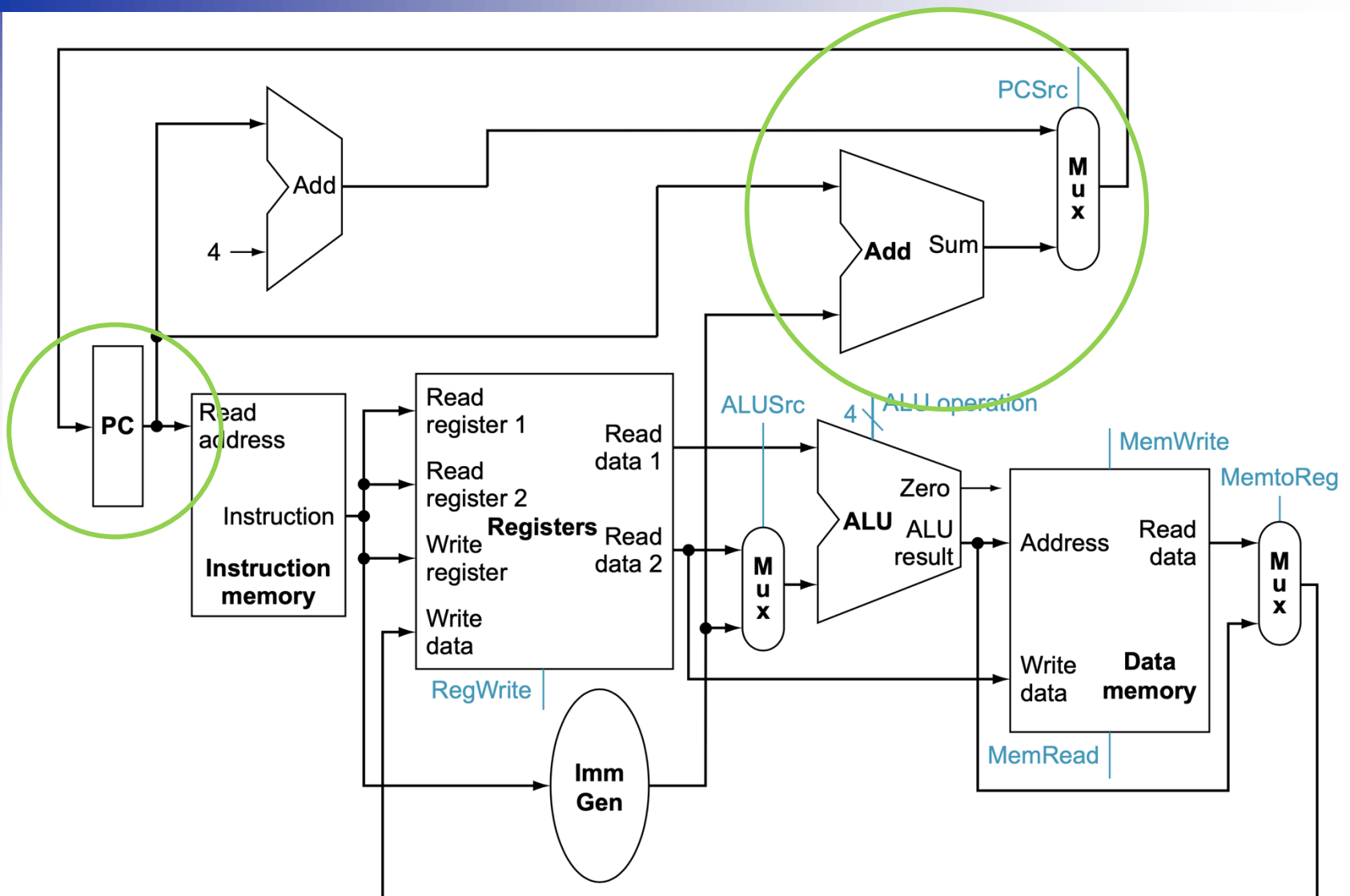
- RISC-V calling convention requires the stack pointer to be **16-byte aligned** at function calls
- Stack frames must preserve this alignment
- Proper alignment improves portability and can simplify memory accesses

Unconditional Jump Commands

- **Unconditional jumps are primarily used for:**
 1. Simple jumps, which do not store the return address
 2. Procedure calls, where there is the need to store the return address, a.k.a. “**linking**”

- **Can be relative to the PC or to an absolute address stored in a register**
 - Jump and Link (JAL) – PC-relative jump
 - Jump and Link Register (JALR) – jump to an address computed from a register and an immediate

Big Picture: Building a Processor



Procedure Call Instructions

- **Procedure call: jump and link**

`jal rd, ProcedureLabel`

- Address of following instruction (PC+4) put in `rd` (usually `x1`)
- Jumps to target address

- **Procedure return: jump and link register**

`jalr rd, offset(rs1)`

- Address of following instruction (PC+4) put in `rd` (usually `x0`)
- Copies `rs1+offset` to program counter (`PC = rs1+offset`)

Procedure Calling

```
int proc(int arg1, int arg2){ // arguments in x10 and x11
    int r = ...;              // r in x18, need to save on stack
    ...
    return r;                 // return value in x10
}
```

```
_main:
    ...
    li    x10, ...            # put argument in x10
    li    x11, ...            # put argument in x11
    jal   x1, proc            # jump and link
    ...

proc:
    addi   x2, x2, -16         # adjust stack for saving x18 (aligned)
    sw     x18, 0(x2)         # save x18 on stack
    ...
    lw     x18, 0(x2)         # restore x18 from stack
    addi   x2, x2, 16         # deallocate stack space
    jalr   x0, 0(x1)         # return to calling routine
```

Using Pseudo-instructions

- **call** simplifies calling a function by handling jump and return address setup
 - Saves the return address in the `x1 (ra)` register and jumps to the address of `function_name`

- **Usage:**

```
call function_name
```

- **Translates to:**

```
jal ra, function_name          # nearby function
```

- Or:

```
auipc ra, offset              # far away function  
jalr ra, offset(ra)
```

Using Pseudo-instructions

- **ret** provides a simple way to return to the calling function
 - Jumps to the address stored in the `x1 (ra)` register

- **Usage:**

```
ret
```

- **Translates to:**

```
jalr x0, 0(ra)
```

Procedure Calling Revisited

```
int proc(int arg1, int arg2){ // arguments in x10 and x11
    int r = ...;              // r in x18, need to save on stack
    ...
    return r;                  // return value in x10
}
```

```
_main:
    ...
    li    x10, ...             # put argument in x10
    li    x11, ...             # put argument in x11
    call  proc                 # jump and link (return address in x1)
    ...

proc:
    addi  x2, x2, -16           # adjust stack for saving x18
    sw    x18, 0(x2)           # save x18 on stack
    ...
    lw    x18, 0(x2)           # restore x18 from stack
    addi  x2, x2, 16           # deallocate stack space
    ret                               # return to calling routine (return address in x1)
```

Register Names

- Using **register names** (e.g., `t0`, `s1`, `a0`) instead of numbers (e.g., `x5`, `x9`, `x10`) makes **code easier to read and understand**
- **Names suggest the register's purpose**
 - Temporary registers, saved registers, function argument registers, ...
- **Most modern RISC-V assemblers, such as GNU Assembler (GAS), provide support for these mnemonic register names**
 - Mapping them to their corresponding register numbers

Register Names

- **a0 – a7**
 - Arguments (x10 – x17)
- **a0, a1**
 - Return values (x10 – x11)
- **t0 – t6**
 - Temporaries (x5 – x7, x28 – x31)
 - **Can be overwritten by callee**
- **s0 – s11**
 - Saved registers (x8 – x9, x18 – x27)
 - **Must be saved/restored by callee**

Register Names

- **gp**
 - Global pointer for static data (x3)
- **sp**
 - Stack pointer (x2)
- **fp**
 - Frame pointer (x8)
- **tp**
 - Thread pointer (x4)
- **ra**
 - Return address (x1)

Procedure Calling Revisited

```
int proc(int arg1, int arg2){ // arguments in a0 and a1
    int r = ...;              // r in s2, need to save on stack
    ...
    return r;                  // return value in a0
}
```

```
_main:
    ...
    li    a0, ...              # put argument in a0
    li    a1, ...              # put argument in a1
    call  proc                  # jump and link (return address in ra)
    ...

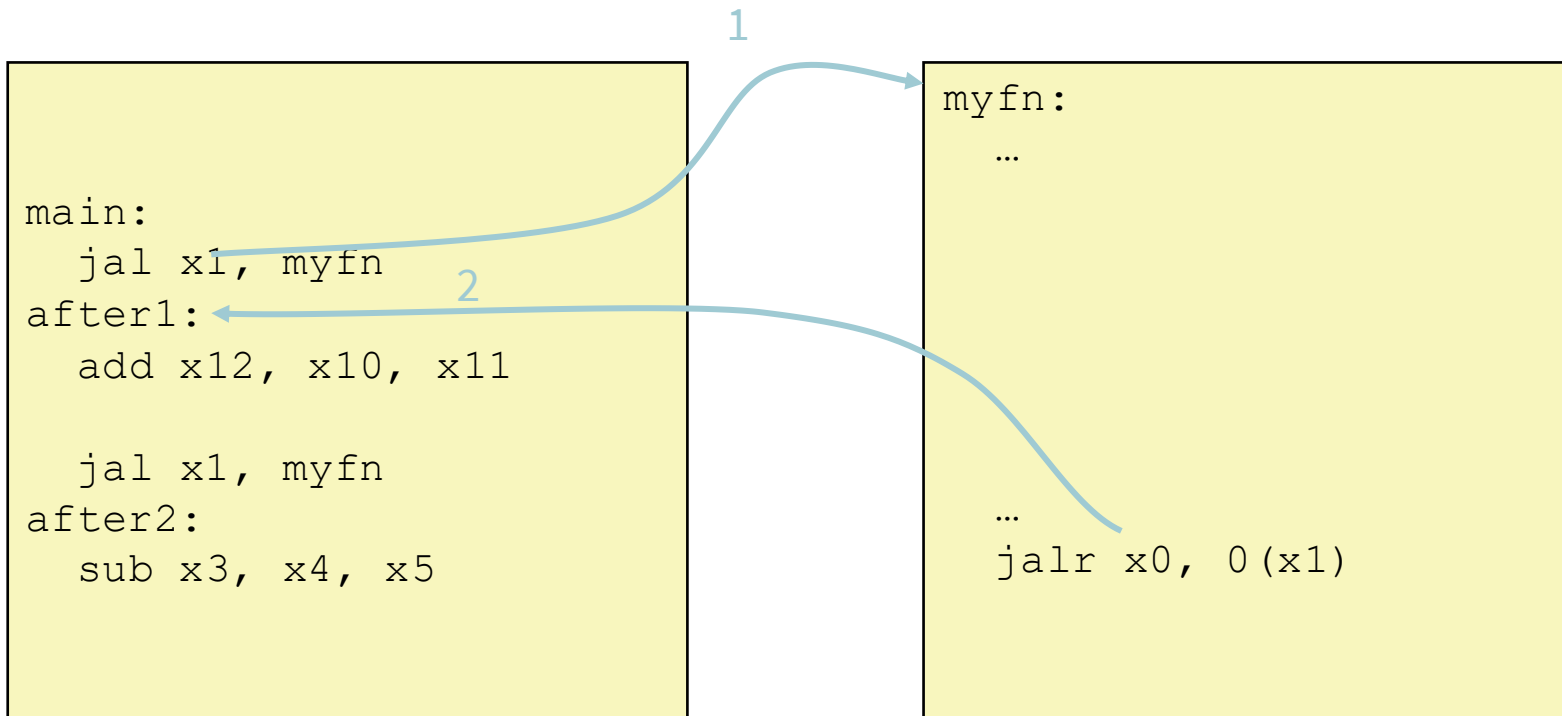
proc:
    addi  sp, sp, -16           # adjust stack for saving s2
    sw    s2, 0(sp)             # save s2 on stack
    ...
    lw    s2, 0(sp)             # restore s2 from stack
    addi  sp, sp, 16            # deallocate stack space
    ret                          # return to calling routine (return address in ra)
```

Takeaway

First call

x1

after1

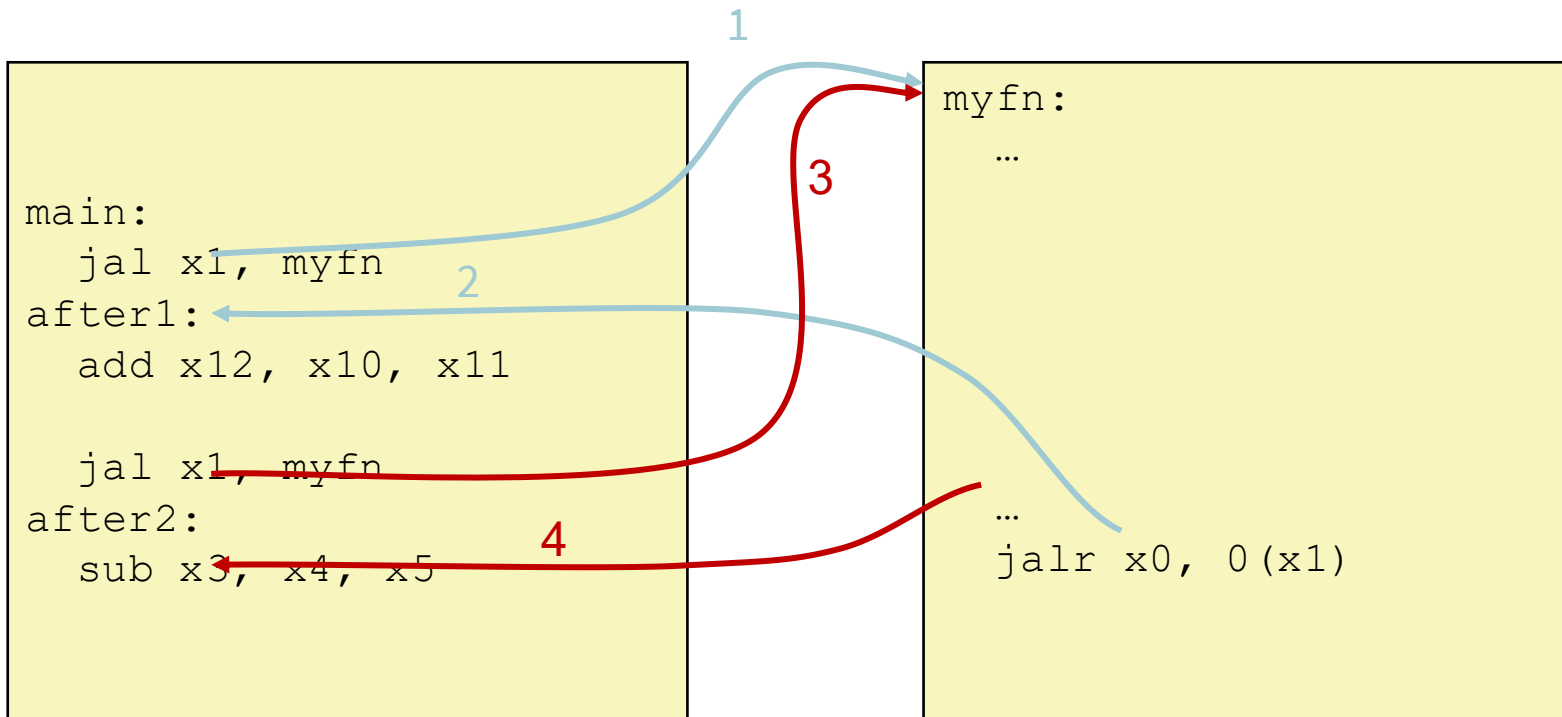


Takeaway

Second call

x1

after2



Takeaway

- **JAL (jump and link) transfers control to the target address**
 - And simultaneously saves the PC+4 in register `x1` (aka `ra` or return address)
- **JALR (jump and link register) gets back from the subroutine to the instruction immediately following the jump**
 - By transferring control back to PC in register `x1`
 - Can also be used for computed jumps (jump tables)
 - e.g., for case/switch statements

Register Saving Conventions

- Can registers be safely used for temporary storage in both functions?

```
f1:
...
li    t0, 15213
jal   ra, f2
add   t1, t1, t0
...
jalr  x0, 0(ra)
```

```
f2:
...
la    t0, vec
...
addi  t0, t0, 4
...
jalr  x0, 0(ra)
```

- **Contents of register `t0(x5)` overwritten by `f2`**
 - This may cause incorrect behavior unless `t0` is preserved or an alternative register is used
 - We need to preserve caller's state

Register Saving Conventions

■ “Caller save”

- The caller must preserve values it needs after the call
- The caller restores them after returning from the callee

■ “Callee save”

- The callee must preserve the original values of these registers
- The callee restores them before returning to caller

■ Which registers are caller-save or callee-save?

RISC-V Register Conventions

Register	ABI Name	Usage	Preserved by
x0	zero	The constant 0	n.a.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	n.a.
x4	tp	Thread pointer	n.a.
x5 – x7	t0 – t2	Temporaries	Caller
x8	s0/fp	Saved register/Frame pointer	Callee
x9	s1	Saved register	Callee
x10 – x11	a0 – a1	Function arguments/return values	Caller
x12 – x17	a2 – a7	Function arguments	Caller
x18 – x27	s2 – s11	Saved registers	Callee
x28 – x31	t3 – t6	Temporaries	Caller

Caller Side

1. **If it uses registers `ra`, `t0–t6` or `a0–a7` after the call**
 - Save its values before the call in the current procedure frame
2. **The first 8 arguments are put in registers `a0–a7`**
 - Additional arguments are passed on the stack according to the ABI
 - A frame pointer may be used to access stack-based arguments and local data
3. **Execute a `jal` instruction to jump to the callee's first instruction**
 - Saves the return address in `ra`
4. **Restores any values saved before the call and uses the return value**

Callee Side

1. Allocates memory to local data

- Subtracting the required size from `sp`

2. Saves preserved registers

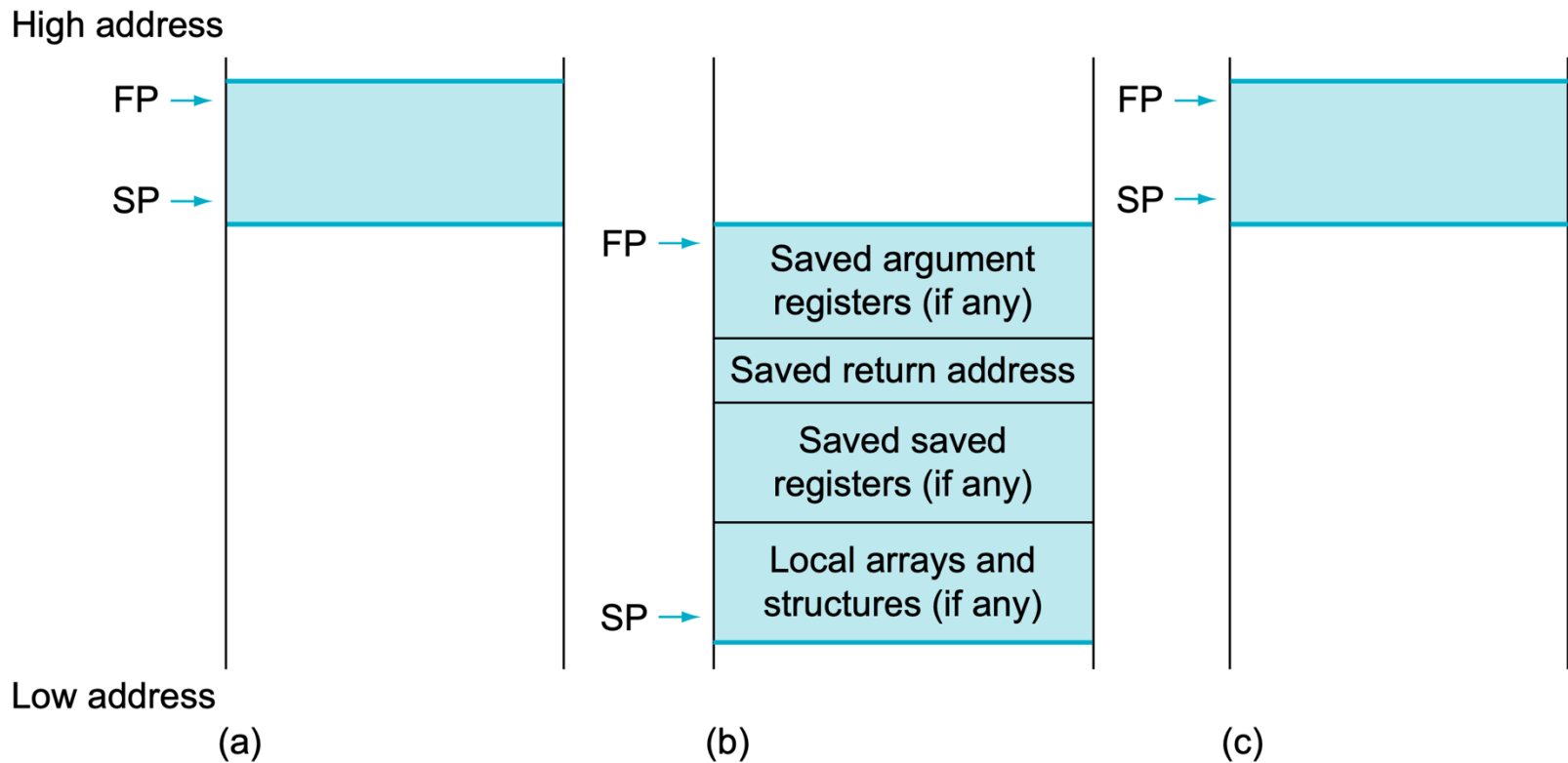
- If it uses registers `s0–s11`, saves its values in the new procedure frame before altering them
- If the callee uses `fp`, it must preserve its previous value
- If the callee makes another call, it must save `ra` before that call

3. Returns control to the caller

- If the callee returns something, put the result(s) in `a0–a1`
- Restore all registers saved in the current frame
- Pop the procedure frame by adding its size to `sp`
- Execute a `jalr` instruction, jumping to the address in `ra`

The Stack and Procedure Call

- The stack allocation (a) before, (b) during, and (c) after the procedure call



Leaf Procedure Example

- **Leaf procedure**

- A procedure that does not call another procedure

- **C code:**

```
int leaf_example(int g, int h, int i, int j){  
    int f;  
    f = (g + h) - (i + j);  
    return f;  
}
```

Leaf Procedure Example

■ RISC-V code:

- Arguments g, ..., j in a0, ..., a3
- f in s0 (hence, **need to save s0 on stack**)
- Result in a0

```
leaf_example:
    addi sp, sp, -16      # allocate stack space
    sw    s0, 0(sp)       # save s0 on stack
    add    t0, a0, a1      # procedure body
    add    t1, a2, a3
    sub    s0, t0, t1
    add    a0, s0, x0      # result
    lw    s0, 0(sp)       # restore s0
    addi sp, sp, 16       # deallocate stack space
    jalr x0, 0(ra)       # return to calling routine
```

String Copy Example

- **C code (naive):**
 - Null-terminated string

```
void strcpy (char x[], char y[]){  
    int i = 0;  
  
    while ((x[i]=y[i])!='\0')  
        i += 1;  
}
```

String Copy Example

■ RISC-V code:

- Addresses of `x` and `y` in `a0`, `a1`
- `i` in `s0`

```
strcpy:
    addi sp, sp, -16      # allocate stack space
    sw   s0, 0(sp)        # save s0 on stack
    add   s0, x0, x0      # i = 0

L1: add   t1, s0, a1      # address of y[i] in t1
     lbu   t2, 0(t1)        # t2 = y[i]
     add   t3, s0, a0      # address of x[i] in t3
     sb    t2, 0(t3)        # x[i] = y[i]
     beq   t2, x0, L2      # exit loop if y[i] == 0
     addi  s0, s0, 1        # i = i + 1
     jal   x0, L1          # next iteration of loop

L2: lw    s0, 0(sp)        # restore saved s0
     addi  sp, sp, 16      # deallocate stack space
     jalr  x0, 0(ra)        # return
```

Non-Leaf Procedures

- **Procedures that call other procedures**
 - Will overwrite important data, such as the return register!
- **For nested call, caller needs to save on the stack:**
 - Its return address
 - Any arguments and temporaries needed after the call
- **After returning**
 - Restore the register state from the stack

Non-Leaf Procedure Example

- **C code:**

- Note that this example assumes the RISC-V M extension (RV32IM)

```
int fact (int n){  
    if (n < 1)  
        return 1;  
    else  
        return n * fact(n - 1);  
}
```

Non-Leaf Procedure Example

- **RISC-V code:**
 - Argument n in $a0$, result in $a0$

```
fact:
    addi sp, sp, -16      # allocate stack space
    sw   ra, 4(sp)        # save return address
    sw   a0, 0(sp)        # save argument

    slti t0, a0, 1        # test for n < 1
    beq  t0, x0, L1        # if so, result is 1
    addi a0, x0, 1        # pop 2 items from stack
    addi sp, sp, 16        # and return
    jalr x0, 0(ra)

L1: addi a0, a0, -1        # else decrement n
    jal  ra, fact          # recursive call
    lw   t0, 0(sp)         # restore original n
    lw   ra, 4(sp)         # and return address
    addi sp, sp, 16        # deallocate stack space
    mul  a0, t0, a0        # multiply to get result
    jalr x0, 0(ra)        # and return
```

Function Call Checklist

■ Caller

1. Save caller-saved registers if needed
2. Place arguments in `a0–a7`
3. Execute `call`

■ Callee

1. Create stack frame
2. Save callee-saved registers
3. Execute function body
4. Place result in `a0/a1`
5. Restore registers
6. Execute `ret`

C Sort Example

- **In the next slides, we derive the RISC-V code from two procedures written in C:**
 - one to swap array elements...
 - ...and one to sort them
- **The example is more complex and combines several concepts from this lecture**
 - For homework, carefully trace the code and identify how the C constructs are translated into RISC-V
 - Your task is to reconstruct the translation step by step

C Sort Example

- **Swap procedure (leaf)**

```
void swap(int v[], int k){  
    int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

The Swap Procedure

■ RISC-V code:

- v in a0, k in a1, temp in t0

```
swap:
    slli  t1, a1, 2          # t1 = k * 4
    add   t1, a0, t1         # t1 = v+(k*4) → (address of v[k])
    lw    t0, 0(t1)          # t0 (temp) = v[k]
    lw    t2, 4(t1)          # t2 = v[k+1]
    sw    t2, 0(t1)          # v[k] = t2 (v[k+1])
    sw    t0, 4(t1)          # v[k+1] = t0 (temp)
    jalr  x0, 0(ra)          # return to calling routine
```

The Sort Procedure in C

■ Non-leaf (calls swap)

- v in $a0$, n in $a1$, i in $s0$, j in $s1$

```
void sort (int v[], int n){
    int i, j;

    for(i = 0; i < n; i += 1){
        for(j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1){
            swap(v, j);
        }
    }
}
```

The Full Procedure

```
sort:  addi sp, sp, -32      # allocate stack space
        sw   ra, 16(sp)      # save ra on stack
        sw   s3, 12(sp)      # save s3 on stack
        sw   s2, 8(sp)       # save s2 on stack
        sw   s1, 4(sp)       # save s1 on stack
        sw   s0, 0(sp)       # save s0 on stack

        ...                  # procedure body
        ...

exit1:  lw    s0, 0(sp)       # restore s0 from stack
        lw    s1, 4(sp)       # restore s1 from stack
        lw    s2, 8(sp)       # restore s2 from stack
        lw    s3, 12(sp)      # restore s3 from stack
        lw    ra, 16(sp)      # restore ra from stack
        addi sp, sp, 32       # restore stack pointer
        jalr  x0, 0(ra)       # return to calling routine
```

The Procedure Body

Move params	<code>addi s2, a0, 0</code>	<code># save a0 into s2</code>
	<code>addi s3, a1, 0</code>	<code># save a1 into s3</code>
Outer loop	<code>addi s0, x0, 0</code>	<code># i = 0</code>
<code>for1tst:</code>	<code>bge s0, s3, exit1</code>	<code># go to exit1 if s0 ≥ s3 (i ≥ n)</code>
Inner loop	<code>addi s1, s0, -1</code>	<code># j = i - 1</code>
<code>for2tst:</code>	<code>blt s1, x0, exit2</code>	<code># go to exit2 if s1 < 0 (j < 0)</code>
	<code>slli t1, s1, 2</code>	<code># t1 = j * 4</code>
	<code>add t2, s2, t1</code>	<code># t2 = v + (j * 4)</code>
	<code>lw t3, 0(t2)</code>	<code># t3 = v[j]</code>
	<code>lw t4, 4(t2)</code>	<code># t4 = v[j + 1]</code>
	<code>ble t3, t4, exit2</code>	<code># go to exit2 if t4 ≥ t3 (pseudo-instruction)</code>
Pass params & call	<code>addi a0, s2, 0</code>	<code># 1st param of swap is v</code>
	<code>addi a1, s1, 0</code>	<code># 2nd param of swap is j</code>
	<code>jal x1, swap</code>	<code># call swap procedure</code>
	<code>addi s1, s1, -1</code>	<code># j -= 1</code>
	<code>jal x0, for2tst</code>	<code># jump to test of inner loop</code>
<code>exit2:</code>	<code>addi s0, s0, 1</code>	<code># i += 1</code>
	<code>jal x0, for1tst</code>	<code># jump to test of outer loop</code>

Takeaway

- **RISC-V divides its instruction set into a base architecture and optional extensions**
 - Each extension is identified by a letter
 - The base integer instruction set is called I
- **RV32I has a relatively small set of instructions**
- **We have already seen most of them:**
 - Arithmetic and logical operations
 - Load data from memory into a register
 - Store data from a register to memory
 - Conditional branches
 - Jump and link for procedure calls

Concluding Remarks

■ Pseudo-instructions

- Simplify assembly programming
- Expand into one or more real instructions

■ The stack

- Provides storage for procedure frames and saved state
- Grows toward lower addresses

■ Calling conventions

- Define how procedures exchange control and data
- Allow independently written procedures to interoperate

Concluding Remarks

- **The ABI provides the rules that allow independently written procedures to work together**
 - `a0–a7` → procedure arguments
 - `a0–a1` → return values
 - `ra` → return address
 - `sp` → stack pointer
 - `t0–t6` → caller-saved registers
 - `s0–s11` → callee-saved registers
- **So far, we have looked at assembly instructions from the programmer's perspective**
 - Next, we will look at what those instructions actually look like to the machine